Warsaw University of Technology
Faculty of Mathematics
and Information Science

# Bachelor's diploma thesis

in the field of study Computer Science and Information Systems

## Design and Implementation of a Collaborative Board Using the Local-First Approach

Piotr Jankiewicz

student record book number 288767

thesis supervisor

dr inż. Paweł Kotowski

WARSAW 2026

**Abstract**

Design and Implementation of a Collaborative Board Using the Local-First Approach

The purpose of this thesis is to design and implement a collaborative board using the local-first approach. The solution uses distribuited conflict free replicated data types (CRDTs) and real-time communication via WebRTC protocol to bring effortless collaboration with other users.
Final result of the effort done is an desktop application that enables drawing and erasing on the whiteboard with mutiple users simultaneously. This paper covers description of solution, but also thouches broader topic of local-first application design architectures.

**Key words: rust language, local-first, CRDT, WebRTC, distributed system, star topology, multithread real time communication**

**Streszczenie**

Projekt i wdrożenie kolaboratywnej tablicy z wykorzystaniem podejścia „local-first"

Celem pracy inynierskiej jest zaprojektowanie i implementacja kolaboratywnej tablicy z wykorzystaniem podejścia "local-first". Wypracowane rozwiązanie uzywa rozproszonego bezkonfliktowego typy danych (CRDTs) oraz komunikacji w czasie rzeczywistym przez protokół WebrRTC by umozliwic bezproblemową kolaborację z innymi uzytkownikami. Ostatecznym wynikiem jest aplikacja desktop-owa umozliwiająca rysowanie oraz zmazywanie poprzednich pociągnięć na białej tablicy z wieloma uzytkownikami jednocześnie. Praca pokrywa opis rozwiązania ale równiez porusza bardziej obszerny temat architektur typu local-first w aplikacjach kolaboratywnych.
**Słowa kluczowe:** język rust, local-first, CRDT, WebRTC, system rozproszony, topologia gwiazdy, wielowątkowa komunikacja w czasie rzeczywistym

# Contents

**1. Introduction** . . . . . **11**

**Introduction** . . . . . **11**

1.1. Analysis and description of collaborative local-first applications . . . . . 11

1.2. Local-first vs Cloud-first . . . . . 15

1.3. Why would one implement own collaborative system ? . . . . . 17

1.4. Examples from industry . . . . . 17

1.5. Known architectures for collaborative applications . . . . . 17

1.6. OT vs CRDTs . . . . . 18

1.7. Vision of the System . . . . . 19

1.8. Requirements Specification . . . . . 20

1.8.1. Functional Requirements . . . . . 20

1.8.2. Non-Functional Requirements . . . . . 24

1.9. Business Cases . . . . . 26

1.10. User Interface vision . . . . . 26

**2. Description of the solution** . . . . . **28**

2.1. System Architecture . . . . . 28

2.2. Technology Stack . . . . . 28

2.3. CRDTS Data Model . . . . . 28

2.4. Communication Layer . . . . . 28

2.5. Synchronization Protocol . . . . . 28

2.6. Application Modules . . . . . 29

2.7. User Interface Implementation . . . . . 29

**3. Analysis of the solution** . . . . . **30**

**4. Conclusions** . . . . . **31**

# 1. Introduction

## 1.1. Analysis and description of collaborative local-first applications

Collaborative applications have not always been so popular as they are these days. You might not even know that some of applications that we use every day are collaborative. In everyday life of a software developer almost all of modern IDEs have collaborative work options, almost all project management software are also collaborative. These are examples close to software engineers, but these applications do not limit themselves to software development. Google Docs, Microsoft 365, Figma, Miro and many more are examples of collaborative applications that are used by millions of people every day. These applications are not only used for work, but also for education and entertainment.

It hasn't always been like this. The steep rise of collaborative applications began as the remote work became more popular but also it owes its rise of popularity to universality of fast global network and increase of computing power of everyday devices.

Current collaborative apps couldn't exist 20 years ago. They haven't been simply invented yet. Typical application of early days of online applications had a simple architecture. It was a client-server application. The server was responsible for storing data and processing it, while the client was responsible for presenting the data to the user and sending user input to the server. This architecture had its advantages and disadvantages. It was simple and easy to implement, but it had a single point of failure. If the server went down, the entire application went down. It also had scalability issues, as the server had to handle all the requests from the clients.

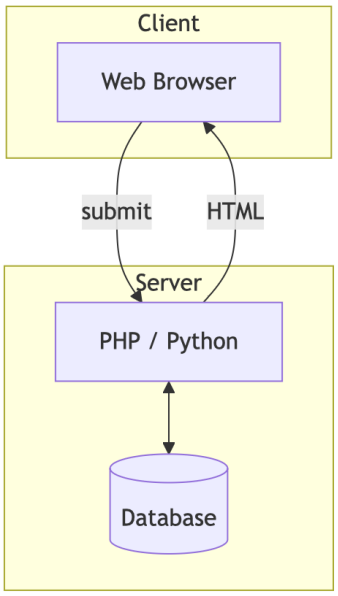


Figure 1.1: Simple client-server architecture of early internet era applications

These kind of application belong to "cloud-first" family. There was no need for "local-first" architectures yet and this term has not been popularized. Before even the discussion on topic of "local-first architectures" the "cloud-first" approach had its own more than one decade of fast development. From simple client server architectures the system rose to more difficult forms of large size client-side application with lots of layers and microservices. Picture below shows architectures that emerged on the market with rise of popularity of mobile and web apps from 2010.

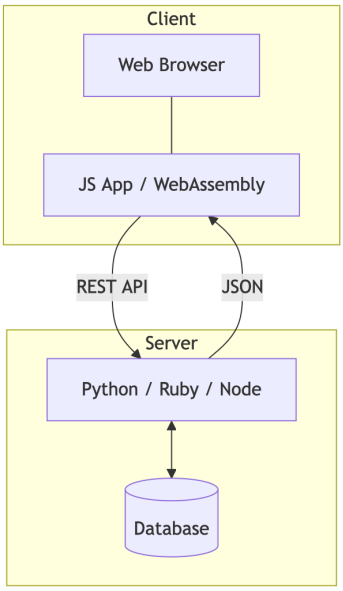


Figure 1.2: More complicated architecture of modern mobile/web applications

How far are these architectures from beeing good fit for collaboarative appliactions? Lets have a look on typical modules that take part in standrd user interface interaction. Picure below shows typical modules of morden web/mobile/desktop cloud-application.

Figure 1.3: Typical modules of modern cloud-based applications

As we can see the level of complication if very high. Simple user intercation with a system triggers multiple functions from multiple modules from multiulpe services. This level of sophistication simply cannot be used for collaborative applications, because orchestration of synchoronization of state shown in simultaneus users would be extremely difficult.

For one, in would require to many asynchronous time-consuming remote calls to different services. Total synchronization nightmare. For some time tough there has been efforts to implement collaboration this way. This approach was known as OT - operational transformation (still used in Google Docs). Although Google Docs is a good example of usage of cloud-first approach, in this thesis I decided to focus on local-first architecutres as they have proved to be more broadly used in the industry and it offers way more possibilities of different kind of applications.

There are also my other reasons to avoid cloud-first architectures when designing collaborative applications. Lets take a look at simple sequance of user interaction with the system. Below we can see general asynchronous interaction of a user with normal cloud-basen application.

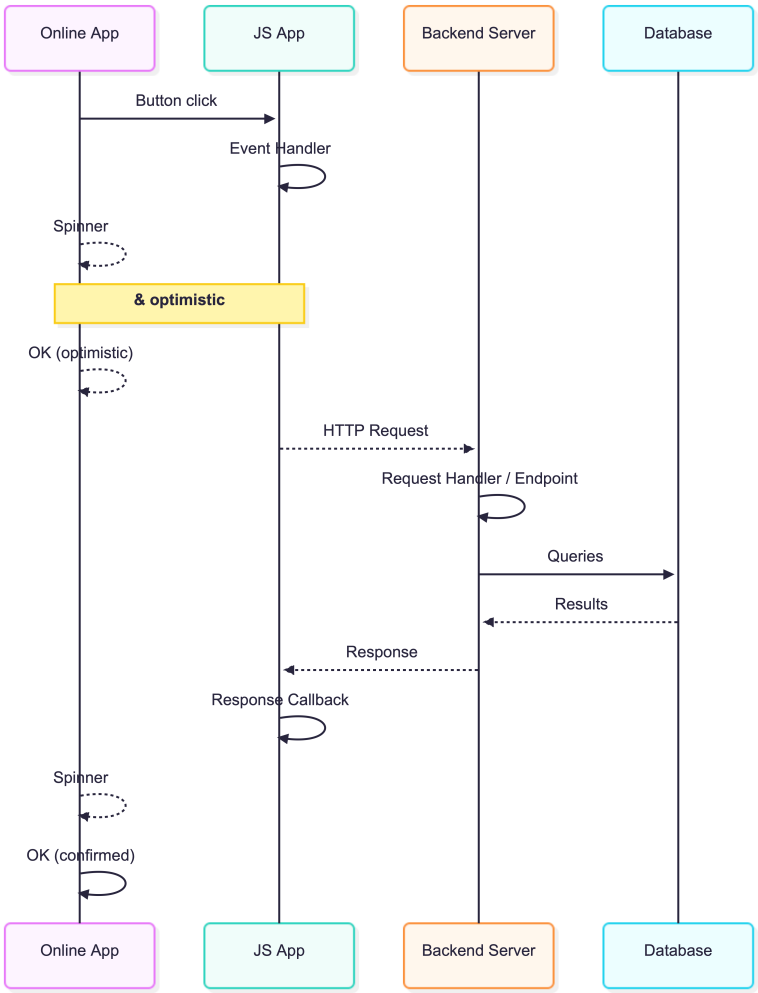


Figure 1.4: Typical sequance of interaction with a user in a modern cloud-based applications

As we can see on on this diagram, the system relies heavily on connectivity with the database. The big question that comes comes to mind is - what happens to the system if there is no internet ? Answer is simple. It would not work. Therefore, one could even say, that whole application is just a asynchronous graphical database wrapper, and it wouldn't be far from truth. This brings us the quastion - What is the definition of cloud-first and local first applications?

In simplest words the definition of cloud-approach is "If the interaction of the user with the system haven't been saved to database, it never happened". On other hand local-first is "What's important the most is local persistance - cloud brings only connectivity".

Therefore let's have a look on how would typical sequance of user interaction with local-first system look. The diagram below shows typical sequance in such system.

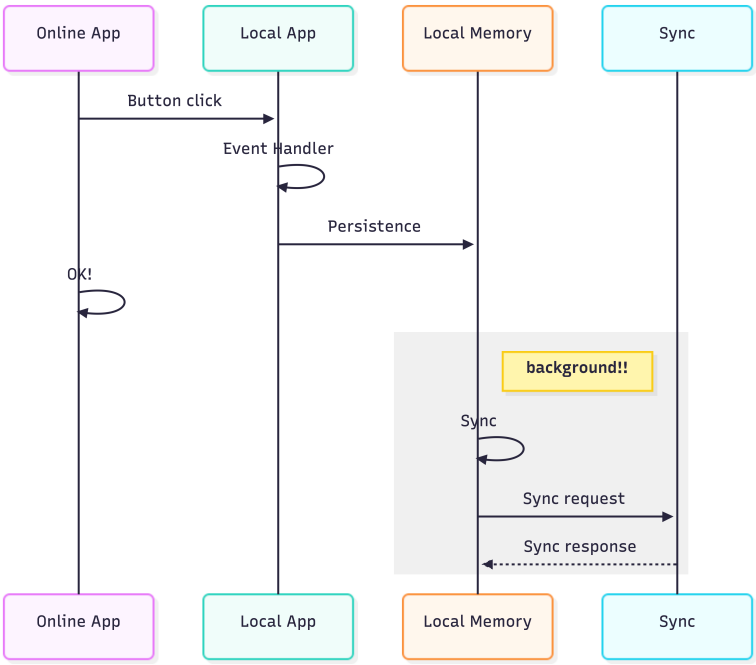


Figure 1.5: Typical sequance of interaction with a user in a local-first applications

The distinction is clear. In local-first approach the user interaction with the system is not dependent on connectivity with the database. Connectivity brings only messaging with other peers. Whole synchronization of replicas of data between users happens on local machine. This is the reaseon why we can call this system distributed. We can also notice that this approach requires much less modules across different serivces. Later, in the thesis, we will clearly how this concept allows us to bring true real time collaboration to the users.

## 1.2. Local-first vs Cloud-first

Lets have a look on the main differences between local-first and cloud-first approaches. The table below shows the main differences between these two approaches when it comes to designing collaborative application.

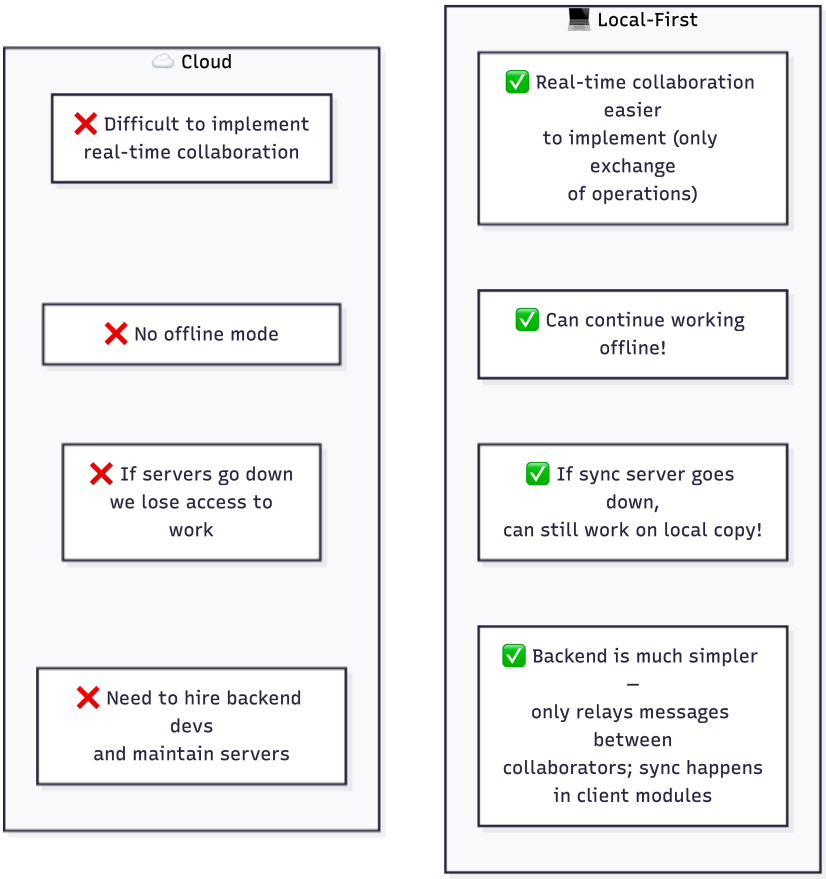


Figure 1.6: Cloud vs Local comparison

Clearly, the obvious choice for apprach when designing collaborative application would be Local-first. Let's now try to distinguish, what are boundaries of such system - what use cases are not applicable with local-first apprach. The table below shows use cases that are not good fit for local-first approach.

Table 1.1: Use cases: Suitable and Unsuitable for Local-First Approach

| **Suitable for** | **Unsuitable for** |
|---|---|
| Real-time collaborative editing (documents, whiteboards) | Applications requiring strong centralized control |
| Offline-first mobile/desktop apps | Systems with strict regulatory or audit requirements |
| Peer-to-peer file sharing | Applications with massive global data aggregation |
| Personal productivity tools with sync | Centralized transactional systems (e.g., banking) |
| Small/medium team collaboration | Apps needing immediate global consistency |

## 1.3. Why would one implement own collaborative system ?

Creating collaborative whiterboard is just an intriduction to the general topic of distrubuted conflict free real time data storage with editable operations. Designign whiteboard is easy exaple for system that can be used to many use cases (distribited conflict free data) - it serves as a great introduction to this topic.

There are multiple use cases/possible usage of distrubuted systems. For example drone swarms - distributed replicas of formation among drones broadcasted between each other. Peer-to-peer file sharing - distributed replicas of files among users. Real time collaborative editing - distributed replicas of documents among users.

## 1.4. Examples from industry

Known examples of local-first applications are: Figma, Miro, Notion, Obsidian, Roam Research, Coda, Google Docs (although it is not pure local-first application). These applications are used by millions of people every day. They are used for work, education and entertainment.

## 1.5. Known architectures for collaborative applications

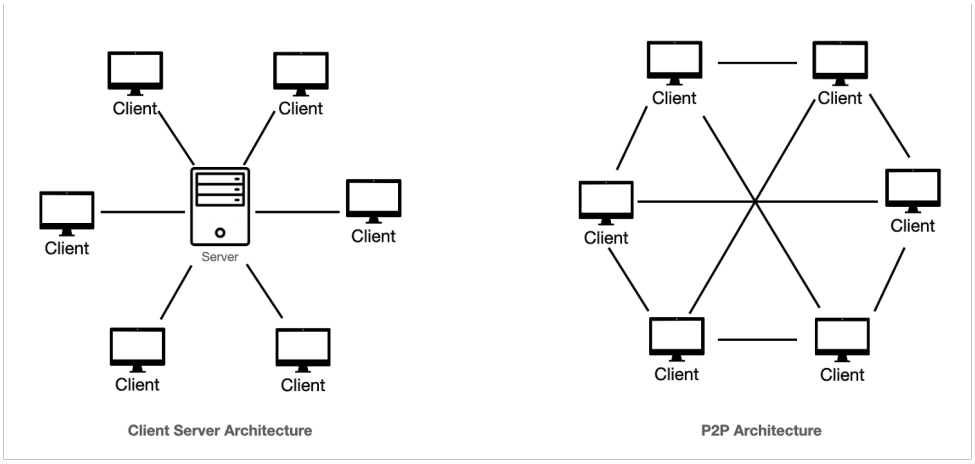


Figure 1.7: Comparison of Client-Server and Peer-to-Peer (P2P) Architectures

Connectivity among multiple users is always solved eigher by peer-to-peer or more "centered" relay architecure for example start-topology. When designig a collaborative system each solution has its benefits and setbacks. In a table belowe there is simple analysis of pros and cons of

choosing one or the ohter when designing local-first collaborative whiteboard.

Table 1.2: Comparison of Peer-to-Peer and Star Topology Architectures

| **Peer-to-Peer (P2P)** | **Star Topology** |
|---|---|
| No need to create a server for communication. Communication is direct - good. Clients manage their connections themselves - might be problematic. | Central node relays communication. It requires support of server. Keeping servers running is expansive. |
| Communication might be faster because it is direct. Users might be close to each other. | Clients sometimes might unnesesarry communicate via very distant server to both of them, which would cause delays. |
| High resilience to single node failure | Central node is a single point of failure |
| Complex synchronization and conflict resolution - corrdination of connections has to be implemented on client side. | Easier management and coordination - server keeps all informations on connectivity. Clients connect only to one server. |
| Scalability can be challenging - many users might have very low maximum of connections it could sustain. | Scalability depends on central node capacity, but server can easily be scaled hirizontlay or verticaly |
| Suitable for decentralized systems with limited numer of peers at the same time (for example 10) | Suitable for medium (up to 50) teams with central coordination - for example video conferance rooms |

As we can see - each solution suits different applications.

## 1.6. OT vs CRDTs

One very important topic in the matter of conflict resolution and collaboration is data structures algorightms that can be used. Local / remote inserts into data structures need to be **commutative**, **associative** and **idempotent**. These three properties are required to ensure that all replicas of data will be eventually consistent - assuming that all operations (inserts/deletes) propagate properly. There are two main approaches to achieve this - OT (operational transformation) and CRDTs (conflict-free replicated data types).

Historically well known solution is OT - operational transformations. The basic idea behind this approach is to transform operations in such a way that they can be applied in any order and still produce the same result. This approach was used in Google Docs and it is still used in

some applications. However, it has some drawbacks. It is complex to implement and it can lead to conflicts that are difficult to resolve.

CRDTs - conflict-free replicated data types - are a more recent approach to achieve eventual consistency in distributed systems. The basic idea behind CRDTs is to design data structures in such a way that they can be updated independently and still converge to the same state. The algorithms are implemented directly into data structures. This approach is simpler to implement and it can handle conflicts more gracefully. It is used in many modern applications, including Figma, Miro, Notion, Obsidian, Roam Research, Coda and many more.

## 1.7. Vision of the System

The envisioned system is a desktop application designed for real-time collaborative drawing on a whiteboard. It leverages the Rust programming language for performance and reliability, and utilizes the egui library to provide a modern, responsive user interface. Communication between users is achieved through WebRTC, with the Livekit open-source server facilitating room-based connections. Each room acts as a channel or document, allowing users to join by simply entering a room code or name, without the need for user accounts, authentication, or authorization.

The application employs CRDTs (Conflict-Free Replicated Data Types) from the Automerge library to ensure seamless synchronization of whiteboard state across all participants. This approach enables users to draw and erase simultaneously, with changes instantly reflected for everyone in the room. The cursors of other users are visible, enhancing the collaborative experience. The whiteboard will be represented as a canvas with raster graphics of strokes from users as vector of points with certain width and color.

For real-time, multithreaded communication, the system uses the Tokio library, ensuring efficient handling of concurrent operations. Notably, the application does not persist data by default; once the app is closed, all data is lost unless explicitly exported to a file by the user. The focus is on desktop environments, with no mobile version planned.

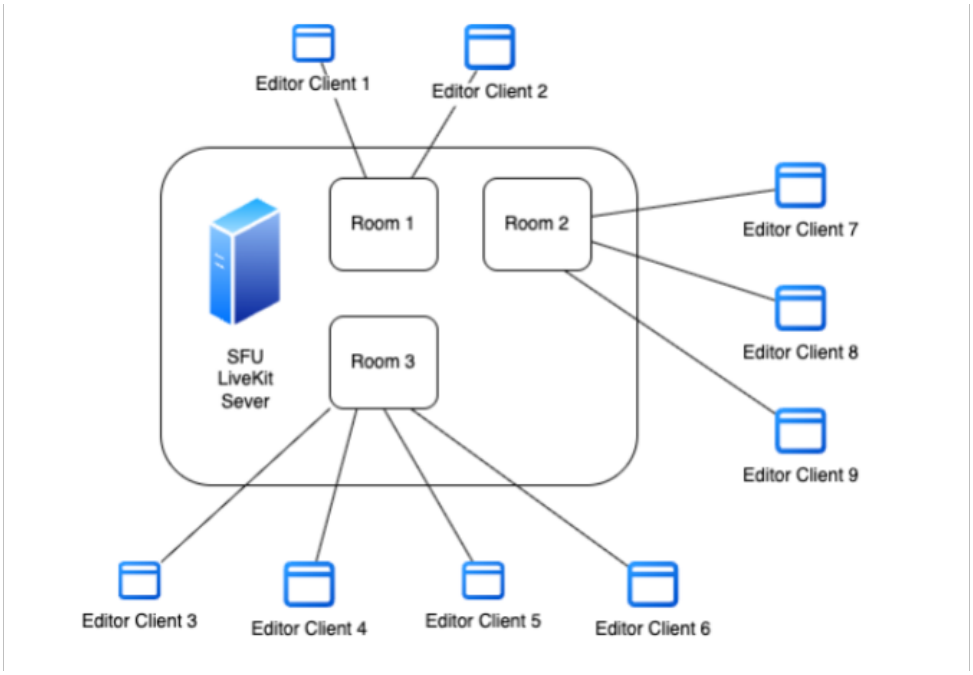


Figure 1.8: Architecture of the envisioned system

## 1.8. Requirements Specification

### 1.8.1. Functional Requirements

Below you can see tables with user stories for each actor in the system: user, application, and server.

Table 1.3: Functional Requirements – User Stories (Actor: User) Part 1

| ID | As a... | I want to... | In order to... |
|---|---|---|---|
| US-01 | User | launch the desktop app and see a blank whiteboard | start drawing immediately without any setup |
| US-02 | User | draw freehand strokes on the whiteboard using a pen tool | express my ideas visually |
| US-03 | User | erase previously drawn strokes | correct mistakes or modify drawings |

Table 1.4: Functional Requirements – User Stories (Actor: User) Part 2

| ID | As a... | I want to... | In order to... |
|---|---|---|---|
| US-04 | User | change the color of the pen | differentiate between elements and add visual clarity |
| US-05 | User | change the thickness (width) of the pen | draw both fine details and bold strokes |
| US-06 | User | clear the entire whiteboard | start drawing from an empty board again |
| US-07 | User | save the current whiteboard state to a file (.crdt/.png) | persist my work for later use |
| US-08 | User | load a previously saved whiteboard document (.crdt/.png) | continue working on a previous drawing |
| US-09 | User | create a new empty document | begin a fresh collaborative session or personal workspace |
| US-10 | User | enter a room name and connect to a collaborative session | work together with other users in real time |
| US-11 | User | disconnect from a collaborative session at any time | leave the room without closing the application |
| US-12 | User | see the cursors of other connected users on the whiteboard | know where collaborators are drawing and coordinate visually |
| US-13 | User | see strokes drawn by other users appear in real time | have a seamless collaborative experience |
| US-14 | User | send chat messages to other participants in the session | communicate textually alongside visual collaboration |
| US-15 | User | be notified when another participant joins or leaves the room | stay aware of who is currently collaborating |
| US-16 | User | join a session without creating an account (only room code needed) | minimize friction and start collaborating instantly |

Table 1.5: Functional Requirements – User Stories (Actor: Application, Part 1)

| ID | As a... | I want to... | In order to... |
|---|---|---|---|
| AP-01 | Application | maintain a local CRDT document (Automerge) representing the whiteboard state | enable offline-capable and conflict-free editing |
| AP-02 | Application | render all strokes stored in the CRDT as raster graphics on a canvas | display the current whiteboard state to the user |
| AP-03 | Application | generate CRDT sync messages when local changes occur | propagate the user's drawing operations to all connected peers |
| AP-04 | Application | receive and apply incoming CRDT sync messages from remote peers | update the local whiteboard with changes made by others |
| AP-05 | Application | establish a WebRTC data channel connection through the LiveKit server | enable real-time, low-latency peer-to-peer communication |
| AP-06 | Application | broadcast the local user's cursor position to all connected peers | allow other users to see where the local user is pointing |
| AP-07 | Application | receive and render remote cursor positions on the whiteboard | display collaborators' cursors with distinguishing color and name |
| AP-08 | Application | run network I/O and CRDT synchronization on separate threads (Tokio) | keep the UI responsive and non-blocking during communication |
| AP-09 | Application | serialize the CRDT document to a byte vector for file export | allow the user to save the full document state |
| AP-10 | Application | deserialize a byte vector from a file into a CRDT document | restore a previously saved whiteboard with full edit history |
| AP-11 | Application | read LiveKit connection credentials from an .env file or environment variables | configure server connectivity without hardcoding secrets |

Table 1.6: Functional Requirements – User Stories (Actor: Application, Part 2)

| **ID** | **As a...** | **I want to...** | **In order to...** |
|---|---|---|---|
| AP-12 | Application | resolve conflicts between concurrent edits automatically via CRDT merge | guarantee eventual consistency without manual conflict resolution |
| AP-13 | Application | represent each stroke as a vector of points with associated width and color in the CRDT | faithfully reproduce strokes across all replicas |
| AP-14 | Application | provide a modern responsive GUI via the egui library | deliver a native desktop experience on macOS and Windows |

Table 1.7: Functional Requirements – User Stories (Actor: Server)

| ID | As a... | I want to... | In order to... |
|---|---|---|---|
| SV-01 | Server | manage rooms (channels) that users can join by name | group collaborating users into logical sessions |
| SV-02 | Server | relay WebRTC data channel messages between all participants in a room (star topology) | ensure every client receives every peer's sync messages |
| SV-03 | Server | authenticate incoming client connections using API key/secret token validation | prevent unauthorized access to collaboration rooms |
| SV-04 | Server | notify all participants when a new user joins a room | allow clients to initiate CRDT sync with the newcomer |
| SV-05 | Server | notify all participants when a user disconnects from a room | allow clients to remove the disconnected user's cursor and update UI |
| SV-06 | Server | automatically create a room when the first participant connects to a new room name | eliminate the need for manual room provisioning |
| SV-07 | Server | support multiple concurrent rooms with independent participant lists | allow several collaborative sessions to run simultaneously |
| SV-08 | Server | forward data channel packets with minimal latency | support the real-time requirements of collaborative whiteboard synchronization |

### 1.8.2. Non-Functional Requirements

Non-functional requirements define the quality attributes of the system. The FURPS+ model is used to categorize them into Functionality, Usability, Reliability, Performance, and Supportability. The table below presents the non-functional requirements of the collaborative whiteboard application.

Table 1.8: Non-Functional Requirements – FURPS+ Analysis

| Category | ID | Description |
|---|---|---|
| **Usability** | NF-01 | The user interface shall be simple and intuitive, allowing basic operations (drawing, erasing, connecting to a session) without the need for training. |
| | NF-02 | The system shall clearly display the synchronization status (online/offline) and inform the user about connection state changes. |
| | NF-03 | The editor shall provide visual indication of collaborators' cursors in real time. |
| | NF-04 | The application interface shall be in English. |
| **Reliability** | NF-05 | The system shall ensure continuity of work in offline mode through local CRDT state, with automatic synchronization upon reconnecting to the session. |
| | NF-06 | The CRDT-based data model shall guarantee that no data is lost during concurrent edits – all user operations shall be preserved after merge. |
| **Performance** | NF-07 | The synchronization time of changes between users shall not exceed 2 seconds on a connection with bandwidth $\geq$10 Mb/s. |
| | NF-08 | Loading a document from local storage (.crdt file) shall not exceed 1 second. |
| | NF-09 | The application shall support simultaneous editing by at least 3 users on one document without noticeable delays in the interface. |
| | NF-10 | The drawing canvas shall maintain a responsive frame rate ($\geq$30 FPS) during normal drawing operations. |
| **Supportability** | NF-11 | The system documentation shall include installation and configuration instructions (README) to enable returning to the project after an extended period. |
| | NF-12 | The application shall be buildable and runnable on macOS and windows. |
| **Constraints (+)** | NF-13 | The application shall be implemented in Rust, using egui for the GUI, Automerge for CRDTs, Tokio for async runtime, and LiveKit for WebRTC signaling. |
| | NF-14 | The application shall not persist any data by default – all state is lost when the app is closed unless explicitly exported by the user. |

## 1.9. Business Cases

Collaboration is neccecary in any business. Companies are build on collaboration and division of work. It is a common effort to build something greater togheter. Therefore having tools that brings real time collaboration is neccecary. The most exiting tools that I think will emarge on the market in coming years are collaborative CAD/CAM applications for engineers. CRDTs might have its best years ahead of them.

## 1.10. User Interface vision

The assumptions for the user interface are as follows: whiteboard canvas takes up the majority of the screen, with a toolbar in the top bar containing pen and eraser tools, color and thickness selectors, and buttons for creating new documents, saving/loading and menu botton.

The menu side panel on the left can be toggled to show connectivity options, including a text field for entering a room/opening connection panel with information on other users and chat. Cursors of other users are shown as colored circles with their ids next to them at the bottom bar. The design is clean and minimalistic, focusing on ease of use and real-time collaboration.

Although the final version of the user interface differs from the initial vision, the main assumptions were met. The first design is shown in the picture below.

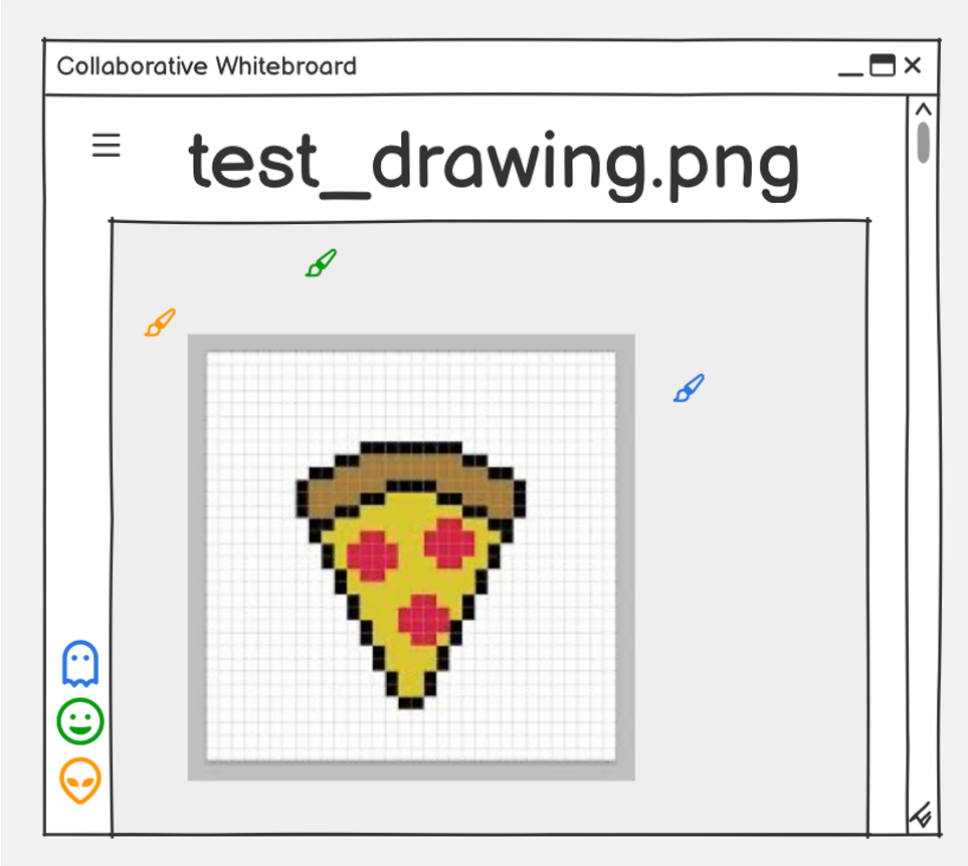


Figure 1.9: Initial vision of the user interface

# 2. Description of the solution

## 2.1. System Architecture

- start topology, livekit SFU, client-side CRDT, WebRTC data channels, - detailes diagrams of architecture - component diagram, egui - automerge - livekit/tokio networking

## 2.2. Technology Stack

- why rust, gui, automerge, tokio, livekit, brief description of each

## 2.3. CRDTS Data Model

- how strokes are represented in CRDT (vectors of points, color, wifth)
- Intent::Draw, Intent::Clear - serialization / deserialization of CRDT document to file

## 2.4. Communication Layer

- livekit room connection, data channel publishing/subscribing, broadcasting of messages/cursor positions, - usage of Tokio.

## 2.5. Synchronization Protocol

automerge sync messages, how they are generated and applied, how conflicts are resolved, how cursors are synchronized, how new users synchronize with existing state when joining a room. "generate_sync_message", "receive_sync_message"

## 2.6. Application Modules

- main.rs... etc - as in slides - dependencies diagrams,

## 2.7. User Interface Implementation

final screenshots, comparison etc

# 3. Analysis of the solution

# 4. Conclusions

# Bibliography


[1] A. Author, *Title of a book*, Publisher, year, page–page.

[2] J. Bobkowski, S. Dobkowski, Title of an article, *Magazine X, No. 7*, year, PAGE–PAGE.

[3] C. Brink, Power structures, *Algebra Universalis 30(2)*, 1993, 177–216.

[4] F. Burris, H. P. Sankappanavar, *A Course of Universal Algebra*, Springer-Verlag, New York, 1981.

## List of symbols and abbreviations

| | |
|---|---|
| nzw. | nadzwyczajny |
| * | star operator |
| ~ | tilde |

If you don't need it, delete it.

# List of Figures


1.1 Simple client-server architecture of early internet era applications . . . . . 12

1.2 More complicated architecture of modern mobile/web applications . . . . . 12

1.3 Typical modules of modern cloud-based applications . . . . . 13

1.4 Typical sequance of interaction with a user in a modern cloud-based applications 14

1.5 Typical sequance of interaction with a user in a local-first applications . . . . . 15

1.6 Cloud vs Local comparison . . . . . 16

1.7 Comparison of Client-Server and Peer-to-Peer (P2P) Architectures . . . . . 17

1.8 Architecture of the envisioned system . . . . . 20

1.9 Initial vision of the user interface . . . . . 27


If you don't need it, delete it.

# Spis tabel

1.1 Use cases: Suitable and Unsuitable for Local-First Approach . . . . . 16
1.2 Comparison of Peer-to-Peer and Star Topology Architectures . . . . . 18
1.3 Functional Requirements – User Stories (Actor: User) Part 1 . . . . . 20
1.4 Functional Requirements – User Stories (Actor: User) Part 2 . . . . . 21
1.5 Functional Requirements – User Stories (Actor: Application, Part 1) . . . . . 22
1.6 Functional Requirements – User Stories (Actor: Application, Part 2) . . . . . 23
1.7 Functional Requirements – User Stories (Actor: Server) . . . . . 24
1.8 Non-Functional Requirements – FURPS+ Analysis . . . . . 25

If you don't need it, delete it.

## List of appendices

1. Appendix 1
2. Appendix 2
3. In case of no appendices, delete this part.